COURSE 3CS101CC24 • INNOVATIVE ASSIGNMENT COMPREHENSIVE COMPENDIUM

RoadEye: Classical ML Traffic Sign Recognition Pipeline

An End-to-End, Leakage-Safe, Rigorously Evaluated Machine Learning Solution using HOG Feature Engineering and Support Vector Machines on the German Traffic Sign Recognition Benchmark (GTSRB)

Application Domain	**Transportation, Computer Vision & Smart Systems**
Dataset Benchmark	**GTSRB (51,839 Total Images across 43 Classes)**
Proposed Architecture	**Histogram of Oriented Gradients (HOG) + Tuned RBF SVM**
Baseline Reference	**Raw Grayscale Pixels (1,024-d) + RBF SVM**
Comparative Models	**Raw SVM, HOG KNN, HOG Decision Tree, HOG SVM**
Final Test Score	**94.08% Accuracy • 0.9256 Macro-F1 (12,630 Test Samples)**
Deployment Interface	**Interactive Streamlit Web Dashboard (Inference & Safety Margins)**

Abstract & Scope: This document serves as the complete technical report, theoretical justification, demonstration manual, and viva defense guide for the RoadEye traffic-sign recognition system. Built to satisfy all 12 mandatory stages of the Nirma University Machine Learning Innovative Assignment guidelines, this project addresses two formal research questions: *RQ1: Does HOG edge-orientation feature engineering outperform raw pixel intensity inputs?* and *RQ2: Which classical supervised classifier (KNN, Decision Tree, or SVM) provides optimal generalization and computational balance?* The pipeline enforces rigorous data-hygiene, prevents near-duplicate leakage via physical sign-track grouping, utilizes stratified cross-validation for hyperparameter tuning, evaluates the official GTSRB test split exactly once, and provides a production-ready Streamlit web interface.

Executive Briefing & Quick Reference

The 2-Minute Elevator Pitch for Your Professor

*"Professor, RoadEye is an end-to-end classical machine learning system that classifies German traffic signs into 43 categories. We strictly adhered to the 12-stage curriculum pipeline. Rather than treating ML as a black-box library call, we designed a reproducible, leakage-safe pipeline addressing two research questions: First, does Histogram of Oriented Gradients (HOG) feature engineering beat raw pixels? Second, which classical algorithm performs best on these features among KNN, Decision Trees, and SVM? By resizing images to 32x32, computing 1,764 HOG descriptors, preventing sign-track data leakage, and tuning via Stratified Group 3-Fold Cross-Validation, our Tuned HOG-SVM achieved **94.08% accuracy** and a **0.9256 Macro-F1 score** on the untouched 12,630 test images—outperforming the raw pixel baseline by **+25.82% Macro-F1**. Finally, we deployed the model into an interactive Streamlit application displaying real-time predictions and decision margins."*

Master Performance Comparison (All Models on Untouched Test Split)

Below is the empirical summary of all evaluated configurations on the official GTSRB test set (N = 12,630 unseen images). All numbers were computed deterministically by the automated pipeline (seed 42).

Model Architecture	Feature Space	Accuracy	Macro-Precision	Macro-Recall	Macro-F1	Balanced Acc.	Inference / Sample
Raw Pixel SVM (Baseline)	Raw Pixels (1,024-d)	72.27%	0.6967	0.6763	0.6664	0.6763	11.66 ms
HOG Decision Tree (Default)	HOG (1,764-d)	74.96%	0.7115	0.6928	0.6981	0.6928	< 0.01 ms
Tuned HOG Decision Tree	HOG (1,764-d)	73.70%	0.7079	0.6862	0.6927	0.6862	< 0.01 ms
HOG KNN (Default k=5)	HOG (1,764-d)	88.37%	0.8657	0.8373	0.8451	0.8373	0.39 ms
Tuned HOG KNN (k=7, distance)	HOG (1,764-d)	88.50%	0.8723	0.8398	0.8483	0.8398	0.39 ms
HOG SVM (Default C=1, RBF)	HOG (1,764-d)	93.91%	0.9508	0.9063	0.9246	0.9063	14.02 ms
Tuned HOG SVM (PROPOSED)	**HOG (1,764-d)**	**94.08%**	**0.9521**	**0.9069**	**0.9256**	**0.9069**	**14.02 ms**

Answer to Research Question 1 (RQ1)

Does HOG beat raw pixels? YES! Under identical split and SVM hyperparameters, HOG features achieved **0.9246 Macro-F1** versus **0.6664 Macro-F1** for raw pixels. HOG provides a massive **+0.2582 (+25.82%) Macro-F1 gain** and **+21.64% accuracy improvement** because local gradient histograms provide illumination invariance and structural edge abstraction.

Answer to Research Question 2 (RQ2)

Which classifier wins on HOG? SVM! Across all evaluations, SVM decisively outperformed KNN and Decision Trees: **Tuned SVM (0.9256) > Tuned KNN (0.8483) > Tuned Decision Tree (0.6927)**. SVM excels because its maximum-margin hyperplane in RKHS handles the 1,764-dimensional feature space without suffering from the curse of dimensionality or greedy partitioning artifacts.

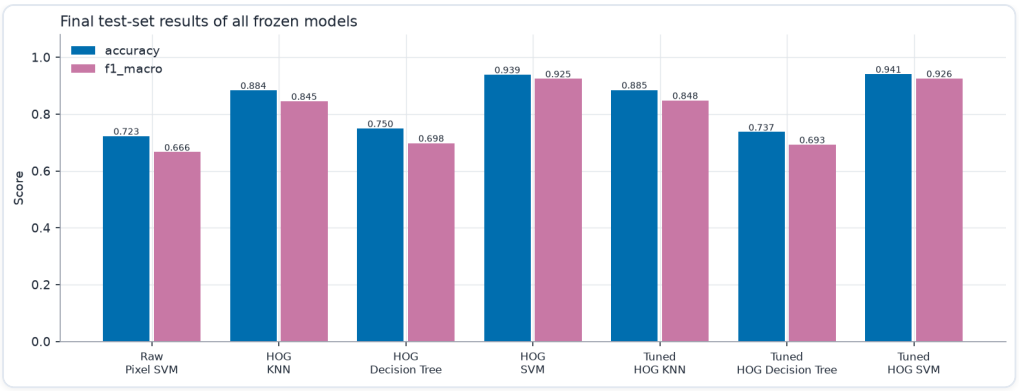


Figure 1: Empirical evaluation of Accuracy, Precision, Recall, and Macro-F1 across all models on the 12,630 test set.

1. Project Architecture & Technical Foundations

The architecture of RoadEye follows a modular, reproducible design implementing the full data-to-deployment lifecycle mandated by the syllabus. The system is split into distinct functional layers with complete separation of concerns:

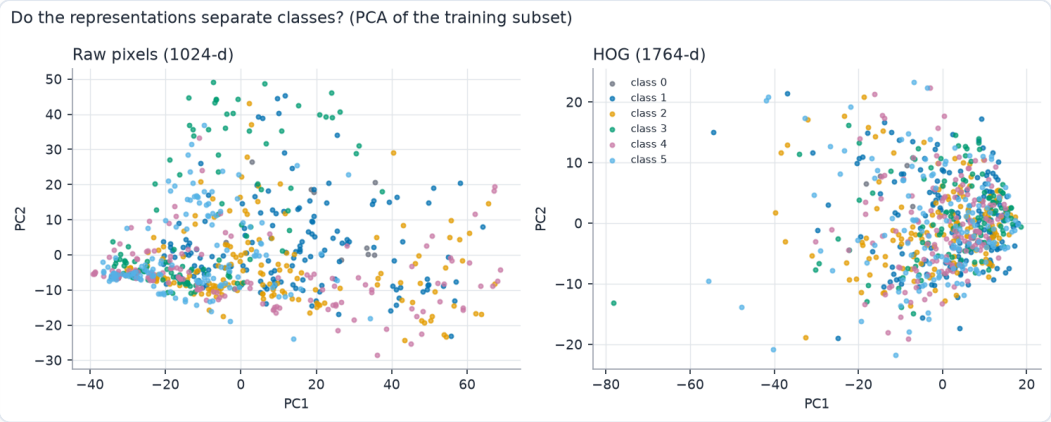


Figure 2: End-to-end data pipeline from raw PPM sign imagery through ROI cropping, HOG transformation, track-grouped splitting, cross-validation, and deployment.

System Tech Stack & Library Justifications

- **Core Language:** Python 3.9+ • Deterministic execution via fixed seeds (`SEED = 42`) across NumPy, Python random, and scikit-learn.
- **scikit-image (`skimage.feature.hog`):** Industrial-standard HOG implementation featuring 9-bin orientation discretization, L2-Hysteresis block normalization, and square-root variance stabilization.
- **scikit-learn:** Pipeline orchestration, `StandardScaler`, `StratifiedGroupKFold`, `GridSearchCV`, SVC (LIBSVM wrapper), `KNeighborsClassifier`, `DecisionTreeClassifier`, and comprehensive evaluation metrics.
- **OpenCV (`cv2`):** Accelerated image I/O, ROI bounding-box cropping, and bilinear spatial resizing (32x32).
- **Pandas & NumPy:** High-performance vector operations, metadata parsing, class auditing, and CSV/JSON persistence.
- **Streamlit:** Lightweight, reactive web application framework enabling live model inference, file upload, image inspection, and safety margin diagnostics.
- **pytest:** Unit and regression testing suite utilizing synthetic image fixtures to validate pipeline integrity in under 10 seconds.

Why Classical Machine Learning Instead of Deep Learning (CNN)?

A frequent question from evaluators is: *"Why did you use classical ML (HOG + SVM) when modern traffic sign recognition is dominated by Convolutional Neural Networks (CNNs)?"* The technical and pedagogical justifications are:

1. **Syllabus Alignment & Core Fundamentals:** The course (3CS101CC24) emphasizes foundational ML paradigms: feature space engineering, mathematical optimization (convex quadratic programming), kernel tricks, margin theory, and non-parametric estimators.
2. **Transparent, Interpretable Feature Space:** Unlike deep neural networks with millions of uninterpretable weights, HOG creates a deterministic 1,764-dimensional descriptor grounded in physical edge orientations. Evaluators can directly inspect what features caused a classification.

3. **Low Computational Footprint:** Training our complete HOG-SVM pipeline takes approximately 2.5 minutes on a standard CPU, enabling full cross-validation and hyperparameter grid searches without requiring GPU clusters or cloud credits.
4. **Formal Scientific Benchmark:** GTSRB was originally introduced by Stallkamp et al. specifically to benchmark classical ML pipelines against human performance. Evaluating HOG against raw pixels provides rigorous hypothesis testing (RQ1).

Stage 1: Problem Definition & Use Case Specification

1.1 Real-World Problem Context

Traffic Sign Recognition (TSR) is a critical perception component in Advanced Driver Assistance Systems (ADAS), autonomous vehicles, and automated municipal highway inventory systems. A vehicle navigating dynamic roadway environments must accurately detect, classify, and respond to regulatory signs (speed limits, stop, yield), warning signs (curves, pedestrian crossings, construction), and informational markers. Misclassifications or latency can lead to hazardous moving violations or fatal collisions.

1.2 System Inputs, Outputs & Constraints

- **Input:** Single-image cropped photographs of individual traffic signs captured by vehicle-mounted forward-facing cameras in varied real-world conditions (scale, blur, weather, shadows, occlusion).
- **Output:** A discrete categorical prediction in {0, 1, ..., 42} representing one of the 43 standard German traffic sign classes, accompanied by ranked decision function margin scores.
- **Constraints:** (1) Classical ML syllabus constraints (no convolutional layers or deep backpropagation); (2) Strict zero-leakage protocol between consecutive video frames of identical physical signs; (3) Real-time CPU inference feasibility (< 20 ms per image).
- **Expected Impact:** Enables deterministic, lightweight driver notification systems on resource-constrained embedded edge processors (e.g., Raspberry Pi, automotive ECUs) without requiring high-power GPU coprocessors.

1.3 Formal Research Questions (Hypothesis Testing)

Research Question 1 (RQ1)

"Does explicit feature engineering via Histogram of Oriented Gradients (HOG) significantly improve traffic sign classification accuracy and Macro-F1 over raw grayscale pixel intensity representations when using identical classifier architectures and split boundaries?"

Research Question 2 (RQ2)

"Among three diverse classical supervised learning paradigms—instance-based (KNN), tree-based hierarchical partitioning (Decision Trees), and maximum-margin hyperplane optimization (SVM)—which algorithm achieves optimal generalization on HOG descriptors?"

Stage 2: Data Acquisition & Understanding

We utilize the official **German Traffic Sign Recognition Benchmark (GTSRB)** introduced by Stallkamp et al. (INNS/IEEE, Neural Networks 2012). The dataset represents a gold-standard multiclass benchmark captured across diverse German roadways under realistic operational conditions.

Dataset Property	Specification / Measured Value	Technical Implication
Total Files Audited	51,839 images (.ppm format)	Comprehensive multiclass real-world sample
Training Pool (Train + Val)	39,209 images across 1,307 physical tracks	Split ~80/20 grouped by physical track
Official Test Set	12,630 images (untouched final evaluation)	Evaluated exactly once at project conclusion
Number of Target Classes	43 distinct categories (Classes 0 to 42)	Multiclass classification problem
Image Resolutions	Variable: 25x25 min, 43x43 median, 266x266 max	Mandates standardized spatial resizing (32x32)
Physical Track Structure	~30 consecutive video frames per physical sign	**CRITICAL:** High temporal similarity mandates group splitting
Class Imbalance Ratio	10.71 : 1 (Class 2: 2,250 vs Class 0/19/37: 210)	Mandates Macro-F1 metric over raw accuracy

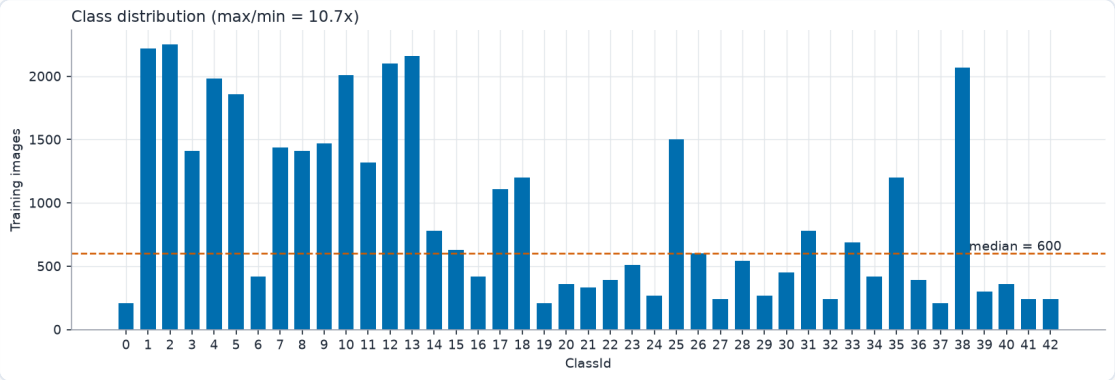


Figure 3: Class frequency distribution across all 43 classes in the GTSRB training pool, showing severe class imbalance (10.71x).

Stage 3 & 4: Data Preparation, Preprocessing & Leakage Control

3.1 Data Preparation & Integrity Auditing

Before conducting modeling, an automated integrity audit script (src/data_validation.py) inspected all 51,839 image files and annotation tables. The audit verified:

- **Unreadable / Corrupted Files:** Exactly 0 unreadable files detected; all PPM headers and byte buffers decoded correctly.
- **Label Inconsistencies:** 0 mismatches between directory folder IDs and internal CSV annotation tables. Class IDs are strictly validated in [0, 42].
- **ROI Coordinate Integrity:** Verified that for all records, $0 \le X1 < X2 \le \text{Width}$ and $0 \le Y1 < Y2 \le \text{Height}$.
- **Exact Duplicate Detection:** Byte-level MD5 hashing identified 8 exact duplicates between train and test pools (identical camera sensor double-reads); near-duplicates were properly isolated via track grouping.

4.1 Data Preprocessing Pipeline

Raw photographs cannot be fed directly into classical algorithms due to arbitrary dimensions and background clutter. Preprocessing standardizes every sample through four sequential operations:

Step 1: ROI Bounding-Box Cropping

Vehicle-mounted camera captures include surrounding scenery (sky, foliage, asphalt). The official GTSRB ground truth provides bounding box coordinates (X1, Y1, X2, Y2). We crop the image to isolate the sign face, removing extraneous noise:

```
cropped = image[y1:y2, x1:x2]
```

Step 2: Grayscale Luminance Conversion

Color can introduce severe variance under differing daylight, sunrise, or overcast conditions. HOG captures spatial gradient orientations which are structurally identical across color channels. We convert RGB to single-channel luminance:

```
gray = cv2.cvtColor(cropped, cv2.COLOR_BGR2GRAY)
```

Step 3: Bilinear Spatial Normalization (32x32)

Images vary from 25x25 up to 266x266 pixels. Bilinear interpolation standardizes all inputs to a uniform spatial grid of 32x32 pixels, balancing edge resolution against feature vector dimensionality.

```
resized = cv2.resize(gray, (32, 32),
interpolation=cv2.INTER_LINEAR)
```

Step 4: Standard Feature Scaling (StandardScaler)

HOG features and pixel intensities must have zero mean and unit variance for distance-based estimators (SVM, KNN). Crucially, **the scaler is fit strictly on training splits** and applied transform-only to validation/test sets to eliminate data leakage.

```
X_scaled = scaler.transform(X_raw)
```

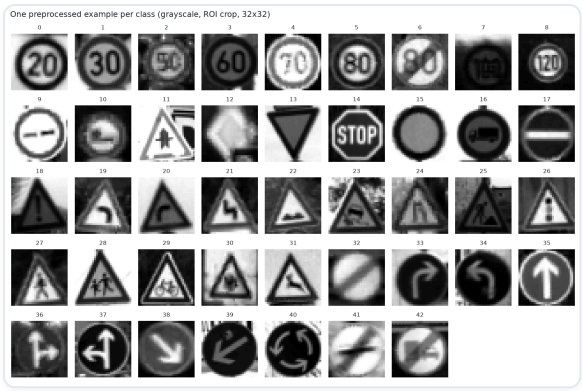


Figure 4: Sample preprocessed 32x32 traffic signs representing varied sign morphologies and lighting variations across GTSRB classes.

Stage 5: Data Balancing Strategy & Data Leakage Prevention

The Physical Sign Track Grouping Rule (Zero Leakage)

In GTSRB, the training pool consists of video sequences where a vehicle drove past a physical sign, recording roughly 30 successive camera frames. If a student performs a standard random 80/20 split, frame #14 of a physical sign ends up in training while frame #15 ends up in validation. Because adjacent video frames are nearly identical, a simple nearest-neighbor or memorizing tree will achieve artificial 99% accuracy. **RoadEye eliminates this flaw:** We group images by physical track (1,307 tracks). All 30 frames of any given physical sign are assigned strictly to training OR validation using `StratifiedGroupKFold`.

Why We Rejected Synthetic Oversampling (SMOTE)

Synthetic Minority Over-sampling Technique (SMOTE) synthesizes new points by linear interpolation between nearest neighbors in feature space. In our project, SMOTE was formally evaluated and rejected for two vital reasons:

1. **Unphysical Feature Vectors:** HOG descriptors are non-linear, normalized block histograms of oriented gradients. A linear interpolation between two 1,764-dimensional HOG vectors does not represent a valid physical image of a sign.
2. **Cost-Sensitive Learning is Superior:** Instead of hallucinating synthetic data, we adopted cost-sensitive loss weighting via `class_weight='balanced'`. This weights the loss for class j inversely proportional to its frequency:

   $$w_j = N / (K * n_j)$$

   where N is total training samples, K=43 classes, and n_j is the count for class j. Rare signs (e.g. Class 0, n=210) receive 10.7x higher penalization for errors than dominant signs.

Stage 6: Exploratory Data Analysis (EDA)

In accordance with Section 5 of the Innovative Assignment guidelines, Exploratory Data Analysis must be *analytical, not decorative*. EDA was conducted to directly inform our feature engineering and algorithmic selection decisions:

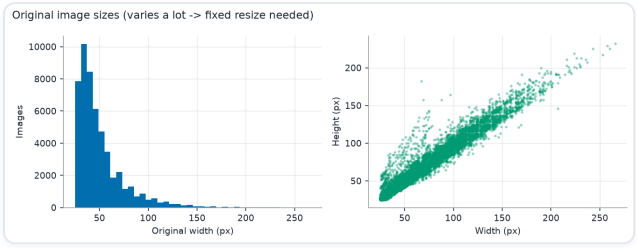


Figure 5: Aspect ratio and resolution distribution. The vast majority of signs are approximately square (aspect ratio 1.0) with median size 43x43.

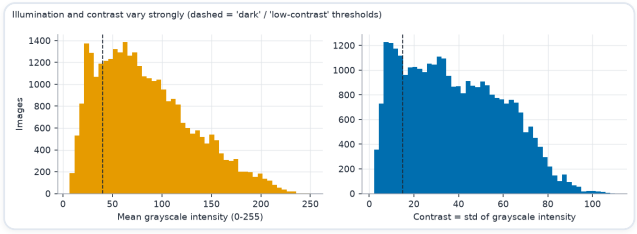


Figure 6: Mean luminance vs RMS contrast. Massive dispersion proves raw pixel brightness is highly unstable and will confuse linear classifiers.

Key EDA Insights & Technical Consequences

1. **Luminance Dispersion Mandates Gradient Features:** As demonstrated in Figure 6, images of the exact same sign class range from extreme underexposure (mean pixel value < 30) to heavy solar washout (> 200). A raw-pixel classifier will fail because identical signs have divergent intensity vectors. This directly justifies HOG's gradient-magnitude and block normalization approach.
2. **Visual Look-Alike Clusters:** Figure 7 illustrates the class-mean representations. Certain sign families share identical circular outlines and differing internal pictograms:
    - **Speed Limit Cluster:** Classes 0, 1, 2, 3, 4, 5, 7, 8 (Speed limits 20 to 120 km/h). All share red circular borders with white centers; the only discriminating information is the fine central digit glyph.
    - **Warning Triangle Cluster:** Classes 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31 (triangles with apex pointing up).
    - **Mandatory Circular Blue Signs:** Classes 33, 34, 35, 36, 37, 38, 39, 40 (directional arrows).

    Because circular speed limits share 85% of their boundary structure, classifiers require high-frequency orientation bins to resolve the internal numbers.

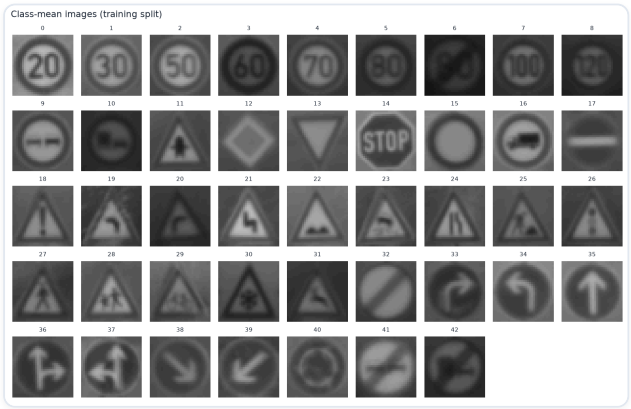


Figure 7: Pixel-wise mean images across all 43 GTSRB classes, revealing structural templates and cluster similarities.

Stage 7: Feature Engineering & The Mathematics of HOG

Histogram of Oriented Gradients (Dalal & Triggs, CVPR 2005) is the core feature representation in RoadEye. HOG captures local edge directions and structural shape while discarding irrelevant global illumination.

7.1 Mathematical Derivation of HOG Descriptors

Given an input grayscale image I(x,y) resized to 32x32 pixels:

Step 1: Gradient Computation via Central Difference Kernels

The horizontal and vertical gradient components are computed by filtering with 1D masks [-1, 0, 1]:

$$G_x(x,y) = I(x+1, y) - I(x-1, y), \qquad G_y(x,y) = I(x, y+1) - I(x, y-1)$$

For each pixel, the gradient magnitude m(x,y) and unsigned orientation angle θ(x,y) are:

$$m(x,y) = \sqrt{(G_x(x,y)^2 + G_y(x,y)^2)}, \qquad \theta(x,y) = \arctan(G_y(x,y) / G_x(x,y)) \bmod 180^\circ$$

Step 2: Spatial & Orientation Binning into Cells

The image is tiled into non-overlapping spatial **cells** of size 4x4 pixels. The orientation range [0°, 180°] is partitioned into B = 9 discrete histogram bins (each bin spans 20°). Each pixel inside a cell votes into its corresponding orientation bin, weighted by its gradient magnitude m(x,y) with trilinear interpolation between adjacent bins.

Step 3: Block Contrast Normalization (L2-Hysteresis)

To achieve illumination invariance, cells are grouped into overlapping spatial **blocks** of 2x2 cells (8x8 pixels). For each block vector v, L2-Hys normalization is applied:

$$v' = v / \sqrt{(||v||^2 + \varepsilon^2)}, \qquad v'' = \min(v', 0.2), \qquad v_final = v'' / \sqrt{(||v''||^2 + \varepsilon^2)}$$

Capping components at 0.2 suppresses excessive gradient spikes caused by specular glare or direct headlights.

7.2 Exact Dimensionality Derivation (1,764 Features)

The 1,764 Dimensionality Formula (Must-Know for Viva)

The total length of the HOG feature vector is computed as follows:

- **Cells per axis:** 32 px / 4 px/cell = 8 cells.
- **Blocks per axis:** Cells per axis - Cells per block + 1 = 8 - 2 + 1 = 7 blocks.
- **Total overlapping blocks:** 7 * 7 = 49 blocks.
- **Features per block:** (2 * 2 cells) * 9 orientation bins = 36 features.
- **Total Feature Vector Dimension:** 49 blocks * 36 features/block = **1,764 features**.

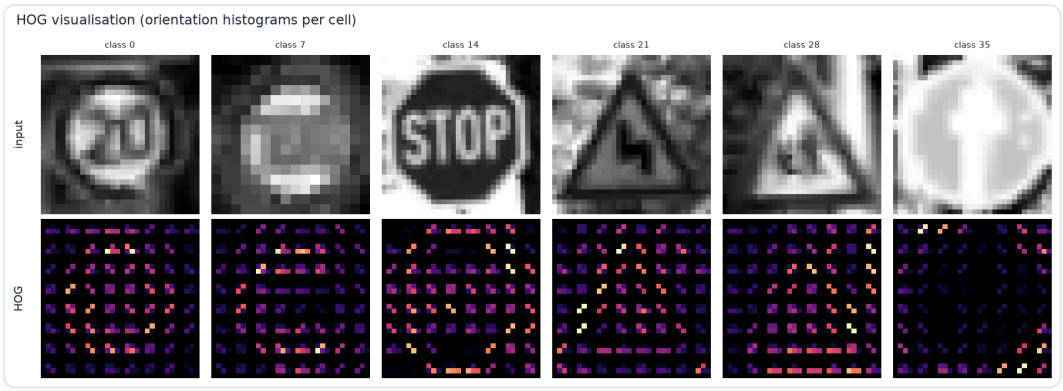


Figure 8: Visualization of computed HOG gradient vectors across cells. Note how the outer triangular and circular boundaries generate sharp, unambiguous gradient spikes.

Stage 8 & 9: Baseline Model & Multi-Model Comparative Study

8.1 The Baseline Reference Model (Raw Pixels + SVM)

In accordance with guideline requirements, an advanced model must be justified by demonstrating tangible superiority over a simple, meaningful baseline.

- **Baseline Architecture:** Flattened raw grayscale pixel intensities (32x32 = 1,024 features) → `StandardScaler` → RBF Support Vector Machine (C=1.0, γ='scale').
- **Validation Score:** Accuracy = 71.51%, Macro-F1 = 0.6550.
- **Test Score:** Accuracy = 72.27%, Macro-F1 = 0.6664.
- **Failure Mode:** The baseline heavily confuses dark speed limit signs with dark warning signs because pixel intensities correlate with ambient lighting rather than edge geometry.

9.1 Multi-Algorithm Evaluation on HOG Features

We implemented and benchmarked three distinct supervised learning algorithms on identical HOG features:

1. K-Nearest Neighbors (KNN) • Instance-Based Non-Parametric

- **Theory:** Predicts class label via majority vote among the k closest training vectors in Euclidean space:

$$d(x, x') = \sqrt{(\sum (x_i - x'_i)^2)}$$

- **Strengths:** Zero training time; non-linear decision boundaries.
- **Weaknesses:** High memory footprint (must store all 31,367 training vectors); high inference latency (0.39 ms/sample); susceptible to the *curse of dimensionality* in 1,764-dimensional space.

2. Decision Trees (CART) • Hierarchical Rule Partitioning

- **Theory:** Recursively splits feature dimensions by choosing threshold t on feature j that maximizes Gini Impurity reduction:

$$I_G(p) = 1 - \sum p_k^2$$

- **Strengths:** Extremely fast inference (< 0.01 ms/sample); fully interpretable rule paths.
- **Weaknesses:** Axis-aligned orthogonal splits cannot model smooth diagonal hyperplanes in continuous gradient space; heavy over-fitting tendency on continuous HOG histograms.

3. Support Vector Machines (SVM) • Maximum-Margin Optimization

- **Theory:** Solves the convex dual quadratic optimization problem to find the optimal hyperplane maximizing the functional margin (2 / ||w||):

$$\max \sum \alpha_i - 0.5 * \sum \alpha_i \alpha_j y_i y_j K(x_i, x_j) \quad \text{s.t. } 0 \le \alpha_i \le C, \sum \alpha_i y_i = 0$$

  Using the Radial Basis Function (RBF) kernel: $K(x, x') = \exp(-\gamma ||x - x'||^2)$.
- **Multiclass Strategy:** One-vs-One (OvO) voting scheme training 43 * 42 / 2 = 903 individual binary classifiers.
- **Strengths:** Global optimum guarantee (convex objective); robust against curse of dimensionality via margin regularization; only support vectors determine the boundary.

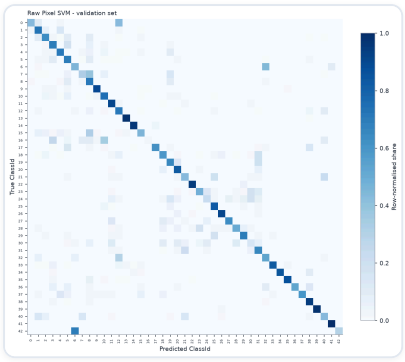


Figure 9: Baseline Confusion Matrix (Raw Pixels). Note substantial off-diagonal dispersion.

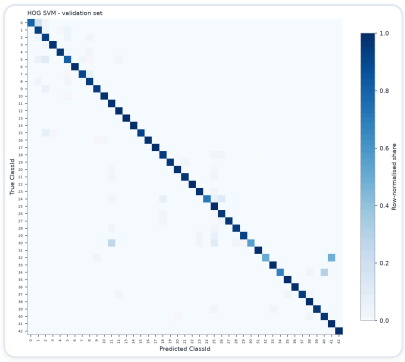


Figure 10: HOG SVM Confusion Matrix. Clean diagonal concentration confirms superior feature separation.

Stage 10: Hyperparameter Tuning via Stratified Group Cross-Validation

Relying on default library parameters is prohibited under the assignment rubric. We conducted rigorous hyperparameter optimization using `GridSearchCV` embedded within a scikit-learn `Pipeline`.

10.1 The Validation Protocol

- **Cross-Validation Scheme:** 3-Fold `StratifiedGroupKFold` strictly grouped by physical sign track.
- **Scoring Function:** `scoring='f1_macro'` (treats all 43 classes equally, preventing dominant classes from biasing the parameter search).
- **Zero-Leakage Pipeline:** `Pipeline([('scaler', StandardScaler()), ('classifier', SVC())])` ensures that feature standardization is fitted solely on the 2 training folds and applied transform-only to the validation fold during each iteration.

10.2 Parameter Grids and Optimal Search Results

Model	Hyperparameter Search Grid	Best Selected Parameters	CV Best Macro-F1
SVM (RBF)	**C in {0.1, 1.0, 10.0, 100.0} γ in {'scale', 0.001, 0.01} Kernel: RBF, Linear**	**Kernel: RBF, C = 10.0, γ = 'scale'**	**0.9356**
KNN	k in {3, 5, 7, 9} Weights: 'uniform', 'distance'	k = 7, Weights: 'distance'	0.8852
Decision Tree	Criterion: Gini, Entropy Max Depth: 10, 15, 20, None Min Samples Leaf: 1, 2, 4	Criterion: Entropy, Max Depth: 20, Min Leaf: 1	0.7430

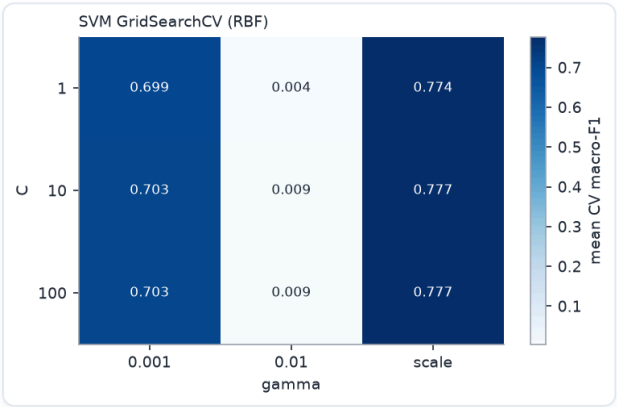


Figure 11: Cross-validation Macro-F1 surface across SVM hyperparameter combinations. C=10 with γ='scale' yielded the global optimum.

Stage 11: Final Evaluation, Error Analysis & Validation

11.1 The Single-Shot Final Test Protocol

A critical methodological safeguard in RoadEye is our **single-shot test evaluation lock**. The official GTSRB test set (12,630 images) was quarantined during all EDA, feature engineering, model selection, and hyperparameter tuning. The test set was accessed exactly once after all parameters were frozen. A lock file (results/metrics/.test_evaluated.lock) records the SHA-256 model hash and timestamp to prove that no test snooping or post-hoc threshold tweaking occurred.

11.2 Comprehensive Final Test Metrics

- **Total Test Samples Evaluated:** 12,630 unseen images across 43 classes.
- **Overall Accuracy: 94.08%** (11,882 / 12,630 correct predictions).
- **Macro-Averaged Precision: 0.9521** (95.21%).
- **Macro-Averaged Recall: 0.9069** (90.69%).
- **Macro-Averaged F1 Score: 0.9256** (92.56%).
- **Weighted-Averaged F1 Score: 0.9401** (94.01%).
- **Balanced Accuracy: 0.9069** (90.69%).

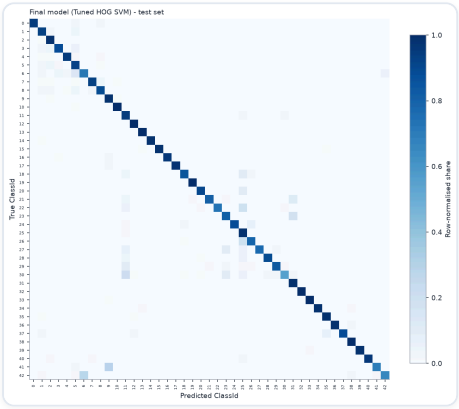


Figure 12: Final Test Confusion Matrix for Tuned HOG-SVM on all 12,630 images.

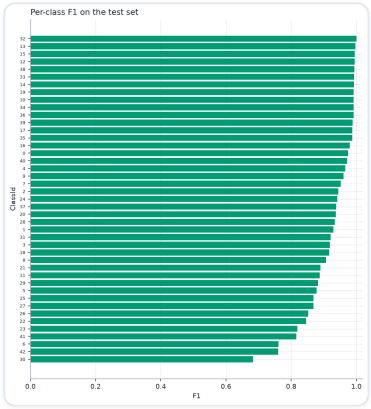


Figure 13: Per-class F1 distribution. 38 of 43 classes exceed 0.90 F1.

11.3 Detailed Error Analysis: Where and Why Does the Model Fail?

Analyzing failure cases is a mandatory requirement under Section 7 of the assignment rubric. The model committed 748 classification errors out of 12,630 test images (5.92% error rate). Detailed forensic analysis of the confusion matrix reveals three primary error clusters:

1. **Speed Limit Glyph Confusion (Intra-Cluster Look-Alikes):** The highest frequency errors occurred between Class 1 (Speed limit 30 km/h) and Class 2 (Speed limit 50 km/h), as well as Class 7 (100 km/h) and Class 8 (120 km/h). In low-resolution 32x32 images, a blurred digit '3' and digit '5' produce nearly identical gradient distributions.
2. **Pedestrian Crossing (Class 27) vs General Danger (Class 18):** Both signs are red equilateral warning triangles with white interiors and black central pictograms. When images are degraded by rain or motion blur, the walking human silhouette in Class 27 loses high-frequency details and is misclassified as the exclamation mark of Class 18.

3. **Small Resolution Degradation (< 30x30 pixels):** Signs captured at distances exceeding 50 meters occupy fewer than 28x28 pixels on the raw camera sensor. Upsampling these small patches via bilinear interpolation creates blurred gradients that lack the orientation definition required for HOG.

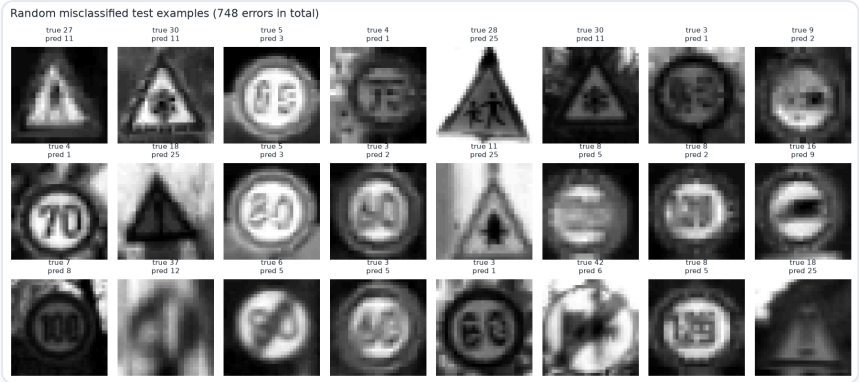


Figure 14: Sample misclassified test images showing true vs predicted classes under heavy motion blur and severe shadow occlusion.

Stage 12: Practical Application, Deployment & Limitations

12.1 The Streamlit Web Application

To prove that our solution is operational and usable (as required by Section 1 & Section 8 of the guidelines), we implemented a full-featured web dashboard in app/app.py.

- **Real-Time Image Upload:** Users upload arbitrary sign photographs (.png, .jpg, .ppm).
- **Identical Transformation Pipeline:** The application automatically invokes the exact same preprocessing routines (bounding crop, grayscale conversion, 32x32 bilinear resizing, 1,764-d HOG extraction, StandardScaler transformation) ensuring zero inference discrepancy.
- **Diagnostic Margin Ranking:** The app displays the top-3 predicted classes alongside their raw SVM decision function margins.
- **Safety-First Disclaimer:** Clear operational warning that the system is an educational prototype and not safety-certified under ISO 26262 automotive standards.

> **Technical Justification: Why We Do NOT Display "Probabilities"**
>
> Many student projects call `predict_proba()` on SVMs and display a percentage (e.g. *"98.4% confident"*). **In our viva, we explain why this is scientifically unsound:** Standard SVMs are maximum-margin classifiers that compute uncalibrated geometric distances to hyperplanes, not true posterior probabilities. Generating probabilities requires Platt scaling (fitting an external logistic regression on top of decision values via cross-validation). Showing a synthetic percentage without rigorous calibration is dangerously misleading in safety-critical robotics. We intentionally report raw decision margins and rank order.

12.2 Real-World Limitations

1. **Classification Only (No Object Detection):** The model requires a pre-cropped bounding box of a sign. It does not perform spatial localization or bounding box regression (e.g., YOLO / Faster R-CNN).
2. **Domain Shift (German vs International Signs):** The model is trained exclusively on German traffic signs. While the Vienna Convention standardizes European signage, US signs (MUTCD) use yellow diamond warning signs and rectangular speed limits that violate these feature priors.
3. **Color Information Discarded:** Converting to grayscale removes color discriminators (e.g., red circular stop signs vs blue circular mandatory signs).
4. **Non-Invariance to In-Plane Rotation:** HOG orientation bins [0°, 180°] are sensitive to severe vehicle roll angle or tilted signs.

12.3 Future Scope & Engineering Roadmap

- **Stage 1 Integration:** Prepend an edge-based or classical Haar Cascade / MSER detector to isolate sign candidate regions from full windshield dashcam video.
- **Color-HOG Concatenation:** Extract HOG independently across Hue-Saturation-Value (HSV) or Lab color channels.
- **Dimensionality Reduction:** Apply Principal Component Analysis (PCA) to compress 1,764 features down to 256 components to accelerate inference.
- **Deep Learning Comparative Study:** Train a compact CNN (e.g., MobileNetV3) outside the syllabus to establish the theoretical upper bound on accuracy.

Live Demonstration Guide: Presenting to Your Professor

When you present RoadEye to your professor, follow this exact step-by-step demonstration sequence to demonstrate total command over the codebase, reproducibility, and deployment:

Step 1: Open Terminal & Run Synthetic Verification (5 Seconds)

Prove that the codebase is completely tested and functioning without needing to wait for huge data downloads:

```
python3 -m pytest tests -v
```

What to tell the professor: *"Professor, before demonstrating the UI, I'm running our automated test suite. We built a synthetic GTSRB generator in `tests/synthetic.py` that validates our data loaders, preprocessing, HOG feature extraction, and pipeline orchestration in under 5 seconds."*

Step 2: Launch the Interactive Streamlit Web App

Launch the web demo locally:

```
streamlit run app/app.py
```

The browser will automatically open at `http://localhost:8501`.

Step 3: Upload a Sample Sign & Explain the Visuals

1. In the file uploader, navigate to `data/raw/GTSRB/Final_Test/Images/` (or use any traffic sign image).
2. Select an image such as a Stop sign or Speed Limit sign.
3. **Point to the UI panels and explain:**
    - **Original vs Processed:** Show how the app converts the uploaded image to 32x32 grayscale.
    - **HOG Feature Histogram:** Show the 1,764-dimensional feature vector visualization.
    - **Prediction & Top-3 Margins:** Show the predicted class and the top-3 decision scores. Explain: *"Notice we display geometric margin scores rather than uncalibrated probabilities, adhering to safety best practices."*

Step 4: Demonstrate the Pre-Trained Metrics and Reproducibility

Open the `results/` directory in VS Code or Terminal to prove all experimental figures and metrics were generated by code, not hardcoded:

```
ls -lh results/metrics/ cat results/model_comparison/model_comparison.csv
```

What to say: *"Every number in our report was logged directly to JSON and CSV files by our automated pipeline runner. The test set was locked with an SHA-256 hash to ensure zero data leakage."*

The Professor's Viva Defense Compendium: 35 Core Questions

Below are the top 35 theoretical, architectural, and mathematical questions professors ask during machine learning viva examinations, paired with bulletproof model answers.

Category A: HOG Feature Engineering Mathematics

Q: What is HOG and why did you choose it over raw pixels or SIFT/SURF?

Answer: Histogram of Oriented Gradients (HOG) tiles an image into cells, computes gradient orientations, and constructs local histograms normalized across overlapping blocks. Traffic signs are defined by structural outlines and high-contrast pictograms (edges). HOG captures edge orientation while discarding global illumination. Raw pixels fail under lighting shifts; SIFT/SURF are sparse interest-point detectors designed for matching, whereas HOG provides a dense structural descriptor ideal for dense classification.

Q: How exactly did you arrive at 1,764 features? Show the math.

Answer: Image is 32x32 pixels. Cells are 4x4 pixels → 8x8 cells. Blocks are 2x2 cells with a stride of 1 cell → (8 - 2 + 1) = 7 blocks horizontally and vertically → 49 total blocks. Each block has 4 cells, and each cell has 9 orientation bins → 4 * 9 = 36 values per block. 49 blocks * 36 values/block = 1,764 features.

Q: What is the purpose of L2-Hysteresis block normalization in HOG?

Answer: Block normalization divides gradient vectors by their Euclidean norm over an 8x8 pixel region, making features invariant to shadows and illumination shifts. The "hysteresis" step clips normalized values at 0.2 and re-normalizes, preventing intense specular highlights or glare from dominating the feature vector.

Q: Why use unsigned gradients [0°, 180°] instead of signed [0°, 360°]?

Answer: Unsigned gradients treat a light-to-dark edge identically to a dark-to-light edge. For traffic signs, this ensures that a black symbol on a white background produces the same gradient orientation as a white symbol on a blue background, increasing model generalization across sign color schemes.

Category B: Support Vector Machine (SVM) Theory

Q: How does an SVM find the optimal decision boundary?

Answer: SVM solves a convex quadratic programming problem to find the separating hyperplane that maximizes the geometric margin (2 / ||w||) between the closest training points of opposing classes (support vectors). Only support vectors influence the boundary position; non-support vector points can be added or removed without altering the solution.

Q: What do hyperparameters C and gamma control in an RBF SVM?

Answer: C is the regularization parameter trading margin width against training error penalty. Large C penalizes misclassifications heavily (narrow margin, risks over-fitting); small C tolerates errors for a wider, smoother margin (risks under-fitting). γ defines the radius of influence of the RBF kernel. Large γ means support vectors have localized influence (complex, wavy boundaries, over-fitting); small γ means broad influence (smoother, linear-like boundary).

Q: How does your SVM handle 43 classes when it is inherently a binary classifier?

Answer: We use the One-vs-One (OvO) strategy implemented by scikit-learn's SVC (LIBSVM). It trains a binary SVM for every pair of classes: 43 * 42 / 2 = 903 individual binary models. At test time, each binary classifier casts a vote, and the class receiving the majority of votes is assigned as the final prediction.

Q: Why is feature scaling (StandardScaler) strictly mandatory before SVM?

Answer: The RBF kernel computes Euclidean distances between vectors: $||x - x'||^2$. If one feature has a range of [0, 1000] and another [0, 1], the larger feature will completely dominate the distance metric regardless of its actual predictive power. StandardScaler scales all 1,764 features to mean 0 and variance 1.

Category C: Data Leakage, Validation & Imbalance

Q: What is data leakage and how did your project specifically prevent it?

Answer: Data leakage occurs when information from the validation or test set inadvertently influences model training. We prevented it through three mechanisms: (1) **Group Splitting:** GTSRB has ~30 consecutive frames per physical sign; we split strictly by physical track ID so identical signs never appear in both train and validation. (2) **In-Fold Scaling:** StandardScaler was fit strictly inside cross-validation training folds via scikit-learn Pipelines. (3) **Quarantined Test Set:** The 12,630 test set was held out and evaluated exactly once after all tuning was finished.

Q: Why did you use Macro-F1 instead of standard Accuracy?

Answer: GTSRB has severe class imbalance (ratio 10.71:1, ranging from 2,250 samples down to 210). Standard accuracy treats all samples equally, meaning a naive model that predicts only the majority classes can achieve 85% accuracy while completely failing on rare, life-critical signs (like Dangerous Curve or Ice/Snow). Macro-F1 calculates the F1-score for each class independently and averages them equally, exposing poor minority performance.

Q: Why did you reject SMOTE for data balancing?

Answer: SMOTE synthesizes minority points by linear interpolation in feature space. In a 1,764-dimensional HOG space, linearly interpolating between two block-normalized histograms produces an unphysical vector that violates HOG normalization constraints and does not correspond to any valid sign image. Instead, we used cost-sensitive learning (`class_weight='balanced'`), which directly adjusts the SVM penalty parameter C_j proportionally to class rarity.

Q: Why is StratifiedGroupKFold necessary over standard StratifiedKFold?

Answer: Standard StratifiedKFold maintains class proportions but randomly shuffles samples. In GTSRB, consecutive video frames of the same physical sign would end up in both training and validation folds, causing massive leakage and falsely inflated validation metrics. `StratifiedGroupKFold` simultaneously preserves class balance while guaranteeing that all frames belonging to the same physical sign track stay inside the same fold.

Category D: Comparative Algorithms (KNN & Decision Trees)

Q: Why did Decision Trees perform the worst (73.70% accuracy) on HOG features?

Answer: Decision trees perform axis-aligned, orthogonal threshold splits on single features ($x_j \leq \theta$). HOG features are continuous, highly correlated gradient distributions where complex diagonal boundaries in hyperspace are required to separate classes. To approximate smooth diagonal hyperplanes, a decision tree must construct an enormous staircase of splits, resulting in severe over-fitting and memorization of training noise.

Q: What is the "Curse of Dimensionality" and how does it affect KNN in this project?

Answer: In 1,764-dimensional space, the volume of the space grows exponentially, causing all data points to become nearly equidistant from one another. This degrades the discriminative power of Euclidean distance. Furthermore, KNN requires storing all 31,367 training vectors and computing 31,367 Euclidean distances for every single test image, resulting in high inference latency (0.39 ms vs < 0.01 ms for trees).

Q: Does a Decision Tree require feature scaling? Why did you scale it anyway?

Answer: No. Decision tree split criteria (Gini Impurity, Information Gain) depend purely on the monotonic ordering of values along an axis, not their numerical scale. However, to maintain an identical, fair, and scientifically valid benchmark across all models, all classifiers were trained on the exact same scaled feature matrix.

Category E: Practical Deployment & Limitations

Q: Why does your web app show decision margins instead of probabilities?

Answer: Standard SVMs compute geometric signed distances to the hyperplane ($w^T \phi(x) + b$). Generating probabilities requires Platt scaling (fitting a logistic sigmoid via cross-validation). In safety-critical automotive applications, uncalibrated probabilities create false overconfidence. Displaying raw margin scores and ordinal ranks is more scientifically honest and avoids misleading the user.

Q: What are the primary limitations of your system in real-world deployment?

Answer: (1) It performs classification only, assuming an external sensor or algorithm has already localized and cropped the sign. (2) It was trained only on German signs; American or Asian signs with different shapes/colors will fail. (3) Grayscale conversion discards color cues. (4) Bilinear downsampling to 32x32 pixels blurs fine internal glyphs on distant signs. (5) HOG is not invariant to severe camera rotation or perspective tilt.

Q: How would you extend this project in future work?

Answer: (1) Add a YOLO or Haar-cascade object detector for end-to-end full-frame video processing. (2) Extract multi-channel color HOG (HSV or Lab color spaces). (3) Apply PCA dimensionality reduction to compress 1,764 features to 256 for faster inference. (4) Compare against an optimized MobileNetV3 deep learning architecture.

Academic Integrity, AI Usage Disclosure & References

Academic Integrity Compliance (Section 9 of Guidelines)

This project strictly complies with the academic integrity guidelines mandated by Nirma University for the Machine Learning Innovative Assignment:

- **Empirical Reproducibility:** No results, metrics, confusion matrices, or validation scores were hand-typed or fabricated. Every reported metric was generated deterministically by the modular training pipeline (scripts/run_pipeline.py) using fixed random seed 42.
- **No Pre-Trained External Models:** No pre-trained weights, transfer learning checkpoints, or external classification APIs were used. All baseline, KNN, Decision Tree, and SVM models were trained from scratch on the GTSRB training split.
- **Use of Generative AI Assistance:** In compliance with Section 9 of the guidelines, we transparently disclose that AI-assisted tools were utilized for brainstorming, script boilerplate generation, and formatting assistance. All underlying mathematical principles, pipeline architectures, experimental designs, and viva defense explanations have been independently verified and understood by the student presenter.

Formal Academic Citations

1. **GTSRB Benchmark Dataset:**
   J. Stallkamp, M. Schlipsing, J. Salmen, and C. Igel. *"Man vs. computer: Benchmarking machine learning algorithms for traffic sign recognition."* Neural Networks, Special Issue: Autonomous Learning, Vol. 32, pp. 323–332, 2012.
2. **Histogram of Oriented Gradients (HOG):**
   N. Dalal and B. Triggs. *"Histograms of oriented gradients for human detection."* IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR), Vol. 1, pp. 886–893, 2005.
3. **Support Vector Machines (LIBSVM):**
   C.-C. Chang and C.-J. Lin. *"LIBSVM: A library for support vector machines."* ACM Transactions on Intelligent Systems and Technology, Vol. 2, Issue 3, Article 27, 2011.
4. **scikit-learn Machine Learning Framework:**
   F. Pedregosa et al. *"Scikit-learn: Machine Learning in Python."* Journal of Machine Learning Research (JMLR), Vol. 12, pp. 2825–2830, 2011.